Week 6. Multi-Networking & Secondary Networking

1. 왜 멀티 네트워크인가

- 기본 K8s 모델: Pod는 **인터페이스 하나(eth0) + 네트워크 하나(클러스터 Pod 네트워크)**
- CNI 스펙 자체는 "컨테이너를 하나 이상의 네트워크에 붙일 수 있다"고 정의하지만, kubelet은 CNI 설정을 **하나만** 호출 → 결과적으로 Pod당 기본 인터페이스 1개
- 이 제약을 풀어주는 게 **Multus** (Pod에 NIC를 여러 개 붙이는 메타 플러그인)

> Multus = "CNI를 여러 번 호출해 주는 CNI"
> 클러스터 네트워크(eth0)는 그대로 두고, 필요한 Pod에만 보조 NIC(net1, net2...)를 추가

2. Multus Architecture / CNI Chaining

2-1. CNI 호출 방식 복습

- CNI 플러그인 = 런타임이 **바이너리를 실행**하는 방식
- 입력: 환경변수(`CNI_COMMAND`=ADD/DEL/CHECK, `CNI_CONTAINERID`, `CNI_NETNS`, `CNI_IFNAME`, `CNI_ARGS`, `CNI_PATH`) + stdin의 JSON 설정
- 출력: stdout의 JSON 결과 (생성한 인터페이스, IP, 라우트 등)
- 컨테이너 안 인터페이스는 `(CNI_CONTAINERID, CNI_IFNAME)` 조합이 **유일**해야 함
  → 같은 컨테이너에 `eth0`, `net1`을 각각 붙일 수 있는 근거

2-2. CNI Chaining과 Multus 구분

구분	CNI Chaining (network configuration list)	Multus (multi-attach)
목적	**한 인터페이스**에 플러그인을 이어 붙여 기능 추가	**인터페이스 자체를 여러 개** 추가
결과	인터페이스 1개	인터페이스 N개 (eth0, net1, net2...)

구분	CNI Chaining (network configuration list)	Multus (multi-attach)
동작	ADD는 순서대로, 이전 결과(`prevResult`)를 다음 플러그인에 전달 / DEL은 역순	네트워크마다 CNI를 **독립적으로** 호출, 각각 다른 `CNI_IFNAME`
예	`bridge` → `portmap` → `bandwidth`	`eth0` =기본 CNI, `net1` =macvlan, `net2` =SR-IOV
설정 위치	`.conflist` 파일	기본은 CNI 설정, 보조는 NetworkAttachmentDefinition

- 둘은 **같이 쓸 수 있음**: 보조 네트워크 하나(net1)의 설정을 conflist처럼 체이닝할 수도 있음 (예: macvlan 뒤에 tuning 플러그인)

> **Chaining = 세로(한 NIC에 기능을 쌓기), Multus = 가로(NIC를 늘리기)**

2-3. Multus가 Pod 생성 때 하는 일

- Multus는 노드의 CNI 설정 디렉터리에서 **가장 먼저 호출되는 CNI**로 등록됨
  → kubelet/런타임은 Multus 하나만 호출한다고 생각
- Multus는 그 안에서 다른 CNI들을 **delegate** 호출

흐름

1. kubelet → CRI 런타임이 Pod sandbox 생성 → CNI(=Multus) ADD 호출
2. Multus가 API 서버에서 해당 Pod를 조회 → annotation `k8s.v1.cni.cncf.io/networks` 확인
3. **기본(primary) 네트워크**: 기존 클러스터 CNI(Cilium, Calico, Flannel 등)를 `eth0` 으로 호출
4. annotation에 적힌 **보조(secondary) 네트워크**마다: NetworkAttachmentDefinition(NAD)을 API 서버에서 읽고 → 그 안의 CNI 설정으로 해당 플러그인을 `net1` , `net2` ... 이름으로 호출
5. 모든 결과를 모아 Pod annotation `k8s.v1.cni.cncf.io/network-status` 에 기록
6. 런타임에는 **기본 네트워크의 결과만** 반환 → `kubectl get pod -o wide` 의 IP, `status.podIP` 는 eth0 기준
   - 보조 NIC의 IP는 Pod status의 IP에 안 나옴 → `network-status` annotation에서 확인
   - 보조 네트워크 하나라도 ADD가 실패하면 Pod 전체가 ContainerCreating에서 멈추는 경우가 많음

2-4. Thin plugin vs Thick plugin

구분	Thin plugin	Thick plugin (권장)
구조	노드에 Multus 바이너리 하나, 호출될 때마다 실행되어 처리	`multus-shim` (CNI 바이너리, 얇음) + `multus-daemon` (상주, 실제 처리)
통신	없음 (바이너리가 직접 처리)	shim ↔ daemon이 노드 로컬 unix socket으로 통신
특징	단순, 리소스 적게 씀	메트릭 노출, 확장 기능 지원, 대신 daemon이 상주하며 리소스 사용

- 설치 시 DaemonSet이 노드에 바이너리를 배포하고, 기존 CNI 설정을 읽어 Multus용 설정을 만들어 줌 (예: `00-multus.conf`, 이름이 앞서야 먼저 호출됨)

3. Primary / Secondary Network

구분	Primary (default) network	Secondary (additional) network
인터페이스 이름	`eth0`	기본 `net1`, `net2`... (annotation에서 `@이름`으로 변경 가능)
만드는 CNI	클러스터 CNI (Cilium, Calico, Flannel...)	NAD에 지정한 CNI (macvlan, ipvlan, sriov, bridge...)
모든 Pod에 붙음?	예	아니오, annotation을 단 Pod만
Service / ClusterIP	동작	**동작 안 함** (kube-proxy, Cilium KPR 등은 primary 기준)
NetworkPolicy	동작	**적용 안 됨** (MultiNetworkPolicy 별도)
DNS (CoreDNS)	eth0 경로 사용	보조 NIC는 클러스터 DNS와 무관
IP 할당	클러스터 CNI의 IPAM	NAD의 `ipam` 설정
기본 라우트	eth0	기본은 없음 (명시해야 함)

- 핵심: **K8s 기능(Service, NetworkPolicy, kubelet probe 경로)은 전부 primary를 기준으로 설계**
  - 그래서 보조 NIC로 받는 트래픽은 앱이 그 IP를 직접 알고 접속하는 방식 (Service 추상화 없음)
  - 헬스체크(kubelet probe)도 Pod IP = eth0 쪽으로 감 → 보조 NIC만 쓰는 앱이라도 eth0 경로로 probe가 통과해야 함

4. NetworkAttachmentDefinition (NAD)

- NAD = 보조 네트워크마다 CNI 설정 지정 CRD

- 핵심은 `spec.config`에 **CNI 설정 JSON 문자열**을 그대로 넣는 것

4-1. NAD 예시 (macvlan)

```
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf-1
spec:
  config: '{
      "cniVersion": "0.3.0",
      "type": "macvlan",
      "master": "eth1",
      "mode": "bridge",
      "ipam": {
        "type": "host-local",
        "ranges": [
          [ {
            "subnet": "10.10.0.0/16",
            "rangeStart": "10.10.1.20",
            "rangeEnd": "10.10.3.50",
            "gateway": "10.10.0.254"
          } ]
        ]
      }
    }'
```

- `type`: 호출할 CNI 플러그인 바이너리 이름 (`/opt/cni/bin`에 있어야 함)
- `master`: **노드의** 부모 인터페이스 이름
  → 모든 노드에 같은 이름의 NIC가 있어야 스케줄링한 곳 어디서든 동작
- `ipam`: IP 할당 방식

4-2. Pod에서 붙이기

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-nic-pod
```

```yaml
  annotations:
    k8s.v1.cni.cncf.io/networks: macvlan-conf-1
spec:
  containers:
  - name: app
    image: busybox
    command: ["sleep", "3600"]
```

annotation 표기 방법

표기	의미
`macvlan-conf-1`	같은 네임스페이스의 NAD
`ns-a/macvlan-conf-1`	다른 네임스페이스의 NAD
`macvlan-conf-1@macvlan1`	인터페이스 이름을 `macvlan1`로 지정
`a, b` (콤마 구분)	보조 NIC 여러 개 (net1, net2 순서로 붙음)
JSON 배열	`interface`, `namespace`, `ips`, `mac`, `default-route` 등 세부 지정

```yaml
# 인터페이스 이름 + 기본 라우트 지정
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |
      [{
        "name": "macvlan-conf-1",
        "interface": "macvlan1",
        "default-route": ["10.10.0.254"]
      }]
```

- `default-route`를 주면 기본 라우트가 보조 NIC 쪽으로 바뀜
  → **eth0 경유 트래픽(Service, DNS)이 깨질 수 있음**

4-3. 붙은 결과 확인

```bash
# Pod 안의 인터페이스 확인
kubectl exec multi-nic-pod -- ip -br addr

# Multus가 기록한 네트워크 상태 확인
```

```
kubectl get pod multi-nic-pod \
  -o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-status}'
```

- `network-status`에는 네트워크별 이름, 인터페이스, IP, MAC, default 여부 등이 JSON 배열로 들어감
- `ip -br addr`로 eth0과 net1이 모두 보이면 성공

5. IPAM: 보조 네트워크의 IP는 누가 주는가

- 보조 네트워크는 클러스터 IPAM을 못 씀 → NAD의 `ipam`에 지정한 플러그인이 할당

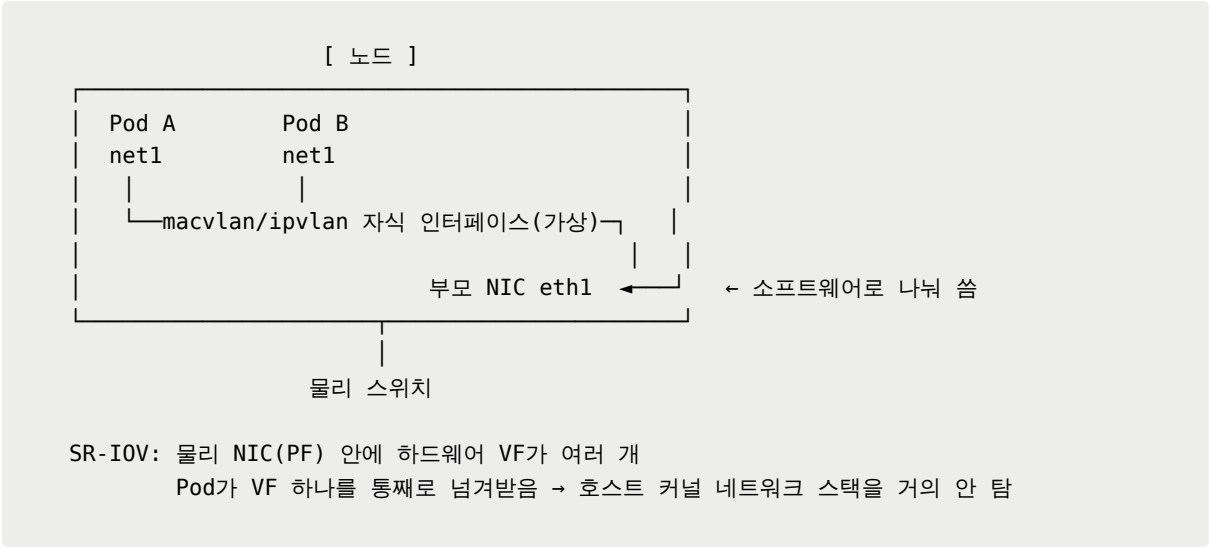
IPAM	범위	특징	주의
`host-local`	**노드 단위** (각 노드가 로컬 파일에 사용 IP 기록)	가장 단순	노드끼리 서로 모름 → 같은 range를 여러 노드에 쓰면 **IP 충돌 가능**
`whereabouts`	**클러스터 전체**	사용 IP를 CRD(K8s 리소스)에 저장해 노드 간 중복 방지, `range`, `exclude` 지원	별도 설치 필요
`dhcp`	외부 DHCP 서버	물리망의 기존 DHCP를 그대로 사용	노드에 CNI dhcp 데몬 필요
`static`	지정한 값 그대로	고정 IP	관리 포인트 증가

```
# NAD config 안의 ipam 부분 예시
"ipam": {
  "type": "whereabouts",
  "range": "192.168.2.225/28",
  "exclude": ["192.168.2.229/30"]
}
```

- 물리망/VLAN에 이미 다른 장비가 있는 대역이면 `exclude`로 겹치는 IP를 빼 줘야 함

6. macvlan / ipvlan / SR-IOV

6-1. 부모 NIC를 Pod가 어떻게 나눠 쓰는가



6-2. macvlan

- 부모 NIC 위에 **자식 인터페이스를 만들고, 자식마다 고유 MAC 주소**를 부여
  → 스위치 입장에선 Pod가 별개의 물리 장비처럼 보임
- 모드
  - `bridge`: 같은 부모를 쓰는 자식끼리 호스트 안에서 직접 통신 (일반적으로 쓰는 모드)
  - `private`: 같은 부모의 자식끼리 통신 차단
  - `vepa`: 모든 트래픽을 외부 스위치로 보냈다가 돌려받음 (스위치가 hairpin 지원 필요)
  - `passthru`: 자식 하나가 부모 NIC를 독점
- 장점: 설정이 쉽고, 기존 L2 망/DHCP/VLAN과 바로 붙음
- 주의
  - **호스트 ↔ 자식(Pod) 직접 통신이 안 됨** (커널 설계상 부모 인터페이스와 macvlan 자식은 서로 못 봄) → 같은 노드의 호스트에서 Pod 보조 IP에 접속하려면 별도 macvlan shim 인터페이스 같은 우회가 필요
  - 스위치 포트가 **한 포트에서 여러 MAC**을 허용해야 함 (포트 보안, 클라우드 가상 NIC에서 막히는 경우 많음)
  - MAC 개수가 늘어나면 스위치 MAC 테이블 부담

6-3. ipvlan

- macvlan과 비슷하지만 **자식들이 부모와 같은 MAC 주소를 공유**하고 IP로만 구분

- 모드
  - `l2`: L2 동작, 같은 서브넷 브로드캐스트 가능
  - `l3`: 부모가 라우터처럼 동작, 서브넷 간 라우팅 (브로드캐스트/멀티캐스트 없음)
  - `l3s`: l3 + 호스트 netfilter/conntrack 거치도록 확장
- 언제 macvlan 대신 쓰는가
  - 스위치/클라우드가 **MAC 여러 개를 허용하지 않을 때**
  - MAC 폭증을 피하고 싶을 때
- 주의
  - 마찬가지로 호스트(부모) ↔ 자식 직접 통신 불가
  - 모든 자식이 같은 MAC → DHCP를 쓸 때 MAC 대신 ClientID로 구분하도록 설정해야 같은 주소가 중복 할당되지 않음(다만, 현재 DHCP IPAM 플러그인이 지원X)

6-4. SR-IOV

- 하나의 물리 NIC(PF, Physical Function)를 하드웨어에서 여러 **VF(Virtual Function)** 로 쪼개는 기술
  → VF를 Pod netns에 통째로 넘김
- 트래픽이 호스트 커널의 소프트웨어 브리지/스택을 거의 거치지 않고 **NIC 하드웨어에서 바로 처리**
  → 지연 낮고 CPU 사용 적음

구성 요소

구성	역할
SR-IOV 활성화	BIOS/펌웨어(IOMMU, SR-IOV), 노드에서 VF 개수 생성 (사전 작업)
`sriov-network-device-plugin`	VF를 K8s **extended resource**(예: `intel.com/sriov_vf`)로 등록 → 스케줄러가 VF 수를 보고 배치
`sriov-cni`	할당받은 VF를 Pod netns로 옮기고 설정
Multus	Pod 생성 시 device plugin이 고른 VF 정보(deviceID)를 sriov-cni에 전달

```
# NAD: SR-IOV 보조 네트워크
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
```

```
  name: sriov-net
  annotations:
    k8s.v1.cni.cncf.io/resourceName: intel.com/sriov_vf
spec:
  config: '{
      "cniVersion": "0.3.1",
      "type": "sriov",
      "name": "sriov-net",
      "ipam": {
        "type": "host-local",
        "subnet": "10.56.217.0/24"
      }
    }'
```

```
# Pod: NAD annotation + VF 리소스 요청
apiVersion: v1
kind: Pod
metadata:
  name: sriov-pod
  annotations:
    k8s.v1.cni.cncf.io/networks: sriov-net
spec:
  containers:
  - name: app
    image: busybox
    command: ["sleep", "3600"]
    resources:
      requests:
        intel.com/sriov_vf: "1"
      limits:
        intel.com/sriov_vf: "1"
```

- **VF 리소스를 requests/limits에 안 쓰면** 스케줄러는 VF 유무를 모르고 배치
  → 잘못된 노드에서 실패 가능
- 드라이버 모드
  - 커널 드라이버 (iavf 등): Pod 안에 일반 netdev(net1)로 보임 → 일반 소켓 앱 가능

- `vfio-pci` (DPDK 등): Pod 안에 netdev가 **안 보임**, 앱이 DPDK로 직접 NIC 제어 (정보는 `PCIDEVICE_*` 환경변수로 전달)

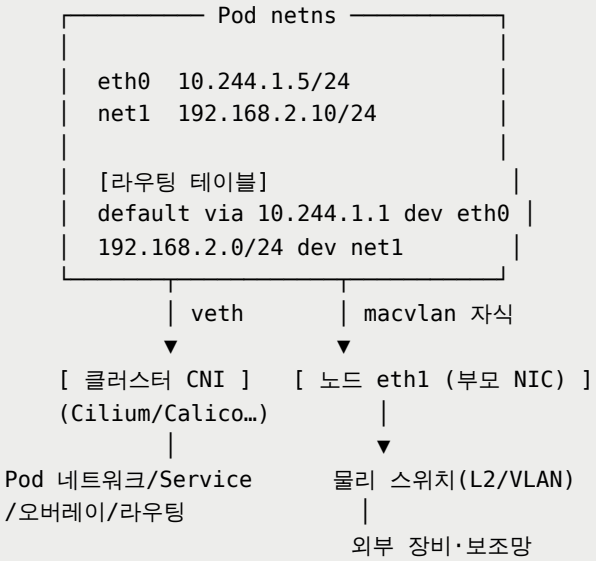
- 주의
  - 하드웨어/펌웨어 의존이 큼 → 노드 사전 준비와 노드 풀 분리 필요
  - VF 개수가 상한 (NIC마다 다름)
  - Pod 이동성·유연성은 낮음 (특정 NIC 달린 노드에 묶임)
  - SR-IOV CNI는 **기본(primary) CNI로 단독 사용 불가** → Multus 필수

6-5. 그 외

방식	설명	용도
`bridge`	노드에 Linux bridge를 만들고 Pod veth를 연결	같은 노드 Pod끼리 L2, 물리 NIC를 브리지에 붙이면 외부와도 연결
`ptp`	Pod ↔ 호스트 veth pair (점대점)	단순한 호스트 연결
`host-device`	호스트의 **기존 인터페이스를 통째로** Pod netns로 이동	VF/물리 NIC 하나를 한 Pod가 독점 (VM 워크로드 등)

7. Multi-NIC Pod 구성 및 Packet Flow

7-1. Pod 내부 모습



7-2. 목적지별 경로

- 인터페이스를 고르는 건 Multus가 아니라 **Pod 안의 일반 리눅스 라우팅** (Multus는 붙이는 것까지만 담당)

목적지	매칭되는 라우트	나가는 NIC	이후 경로
다른 Pod IP, Service ClusterIP, CoreDNS	default route	`eth0`	클러스터 CNI 데이터패스 (Cilium의 eBPF, Calico 라우팅 등) → 정상적인 K8s 통신
보조망 서브넷 (`192.168.2.0/24`) 장비	연결된 서브넷 라우트	`net1`	macvlan 자식 → 부모 `eth1` → 물리 스위치 → 목적지 (호스트 CNI 데이터패스 거치지 않음)
보조망 너머의 다른 대역	별도 라우트 필요	설정에 따라	NAD `routes` 또는 `default-route` 설정이 없으면 eth0로 감

나가는 흐름 (Pod → 보조망 장비)

1. 앱이 `192.168.2.50`으로 전송 → 라우팅 테이블에서 `192.168.2.0/24 dev net1` 선택
2. 출발지 IP는 net1의 IP(`192.168.2.10`), ARP도 net1에서 수행
3. macvlan 자식이 자기 MAC으로 프레임을 만들어 부모 `eth1`로 전달
4. `eth1`이 스위치로 송신 → 목적지 도착
5. **클러스터 CNI(Cilium 등)의 tc/eBPF나 iptables는 이 경로에 관여하지 않음** → NetworkPolicy, Hubble 관측도 안 걸림

들어오는 흐름 (보조망 장비 → Pod)

1. 스위치가 `192.168.2.10`의 MAC(macvlan 자식 MAC)으로 프레임 전달
2. 부모 `eth1`이 수신 → macvlan이 MAC을 보고 해당 자식(Pod의 net1)으로 직접 전달
3. Pod 안에서 net1 수신, 앱까지 도달
4. 응답은 아래 7-3의 **라우팅 대칭성** 규칙에 따라 나감

SR-IOV일 때 차이

- 자식 인터페이스가 아니라 VF 자체가 Pod 안에 있음
- 송수신이 NIC 하드웨어(내장 스위치)를 통해 스위치까지 직행, 호스트 커널의 macvlan/bridge 처리 단계가 없음
- DPDK(vfio-pci)면 커널 네트워크 스택도 건너뛰고 앱이 직접 패킷을 처리

7-3. 비대칭 라우팅

- 상황: 외부 클라이언트(다른 서브넷)가 `net1` IP로 요청 → 응답을 보낼 때 Pod의 default route는 `eth0`
- 결과: 응답이 net1이 아닌 eth0로 나감 (출발지 IP는 net1 IP) → 도착지 방화벽/rp_filter/클러스터 CNI가 드롭하거나, 상대가 응답을 못 받음

해결 방법

- 보조망 대역 라우트를 명시: NAD `ipam`의 `routes` 사용
- 보조 NIC로 기본 라우트를 옮기고 싶으면 annotation의 `default-route` (단, Service/DNS 경로가 깨지지 않는지 확인)
- 출발지 IP 기준으로 라우팅을 나누려면 **source-based routing**
- 노드 쪽 `rp_filter` 설정도 확인 (strict면 비대칭 경로 패킷을 버림)

8. 보조 네트워크의 보안: MultiNetworkPolicy

- 일반 NetworkPolicy는 **primary 네트워크만** 대상
- 보조 네트워크용은 `MultiNetworkPolicy` (CRD, `k8s.cni.cncf.io` 그룹) → 정책이 어느 보조 네트워크에 적용되는지는 annotation `k8s.v1.cni.cncf.io/policy-for`로 지정
- 집행은 별도 구현체가 노드에서 수행 (iptables 또는 tc 기반 구현이 있음)
- 아직 K8s 표준 API가 아니고 구현체 의존적
  → 보조 NIC가 외부망에 직접 노출되는 경우 **물리망 쪽 방화벽/VLAN 분리**를 기본 방어선으로 설계

결론

- Multus: 클러스터 CNI는 그대로 두고, Pod에 필요한 만큼 보조 NIC를 붙여 주는 메타 플러그인. 인터페이스는 네트워크마다 독립 CNI 호출로 생성

Q1. 멀티 네트워크가 필요한 상황

- **트래픽 분리**: 트래픽 경로를 분리 (물리 NIC까지 나눠야 대역폭도 실제로 분리됨)
- **보안/컴플라이언스**: 물리적으로 분리된 망(VLAN, DMZ, 결제망)에 직접 연결해야 할 때
- **기존 L2 망/레거시 연동**: Pod가 기존 서브넷의 "실제 IP"를 가져야 할 때 (IP 기반 ACL, 레거시 시스템, 멀티캐스트/브로드캐스트 필요)
- **고성능/저지연**: 통신사 CNF(5G 코어 등), 패킷 처리 앱(DPDK), HPC, 저지연 트레이딩 → SR-IOV

- **네트워크 기능(NF) 워크로드**: 라우터, 방화벽, 게이트웨이처럼 여러 망 사이를 이어 주는 Pod
- **가상화**: KubeVirt VM에 보조 NIC가 필요한 경우
- 일반적인 웹/API 서비스는 대부분 **필요 없음** → 멀티 네트워크는 Service/NetworkPolicy 같은 K8s 기능을 포기하는 대가가 있으니 필요할 때만 사용

Q2. 각 네트워크 방식이 적합한 사용 사례

방식	핵심 특징	적합한 사례	주의
macvlan	Pod마다 고유 MAC, 물리망에 직접 노출	기존 L2 망/DHCP/VLAN에 Pod를 직접 연결, IP 기반 방화벽 규칙	호스트↔Pod 불가, 스위치/클라우드 MAC 제한
ipvlan	부모 MAC 공유, IP로 구분	MAC 다중 허용이 안 되는 환경(클라우드 가상 NIC 등), MAC 폭증 회피, l3 모드 라우팅	호스트↔Pod 불가, DHCP는 ClientID 고려
SR-IOV	하드웨어 VF를 Pod에 직접 할당	초저지연/고처리량, DPDK, 통신사 CNF	NIC·노드 사전 준비, VF 개수 한계, 이동성 낮음
bridge	노드 Linux bridge에 연결	같은 노드 Pod간 L2, 단순 구성	노드 간 연결은 별도 설계
host-device	호스트 인터페이스를 Pod가 독점	VM 워크로드 등 NIC 한 개를 통째로 전용	확장성 낮음

- 선택 가이드
  1. 성능이 최우선 → SR-IOV (필요 시 DPDK)
  2. 기존 L2 망에 직접 연결 + MAC 다중 허용 → macvlan
  3. MAC 제한이 있는 환경 → ipvlan
  4. 노드 로컬 L2만 필요 → bridge
- 클라우드 VM에서는 macvlan이 막히는 경우가 있어 ipvlan 또는 클라우드 제공 방식(보조 ENI 등)을 먼저 확인

Q3. Pod 내부에 네트워크 인터페이스가 여러 개일 때의 흐름

- **생성 시**: kubelet → 런타임 → Multus(CNI) → ① 기본 CNI로 eth0 생성 → ② annotation의 NAD마다 해당 CNI로 net1, net2... 생성 → ③ `network-status` annotation 기록 → ④ 런타임에는 eth0 결과만 반환
- **통신 시**: 인터페이스 선택은 Pod 안의 **일반 리눅스 라우팅**이 담당

    - Pod/Service/DNS 트래픽 → default route → eth0 → 클러스터 CNI 데이터패스
    - 보조망 서브넷 트래픽 → 해당 서브넷 라우트 → net1 → macvlan/ipvlan/SR-IOV → 물리망 (클러스터 CNI, Service, NetworkPolicy, Hubble 관여 없음)
- **주의**: 보조 NIC로 들어온 요청의 응답이 eth0로 나가는 **비대칭 라우팅** → 라우트 명시, `default-route`, source-based routing으로 해결

질문

1. Pod에 NIC를 여러 개 붙이는 일을 쿠버네티스 자체 기능이 아니라 왜 Multus 같은 외부 플러그인이 맡고 있는지? 단순히 kubelet이 CNI를 하나만 부르는 구조라 그런 건지 다른 이유가 있는지 궁금함.